

AY 2025/2026



POLITECNICO
MILANO 1863

Internet Of Things: Challenge 1

Alberto Biffi - 10677341

Matteo CESANA Professor
Antonio BOIANO Fabio PALMESE

Version 1.0
March 29, 2026

Contents

1	Introduction	2
1.1	Sensors choices	2
2	Other constraint	2
3	IoT Motion Sensor Specification (1.5pt)	3
3.1	Include and Define	3
3.2	Functions	3
3.3	Functions	4
3.4	Setup	4
3.5	Loop	5
3.6	Testing	5
4	Energy Consumption Estimation (1 pt)	6
5	Comment Results and Improvements (2.5 pt)	9
6	Wokwi Links	11
7	Energy Trade-off in a WSN (2pt)	11
7.1	Q1	11
7.2	Q2	11

1 Introduction

We've been asked to create a commercial-style IoT motion sensor for a smart home in Wokwi. The sensor must detect the presence of human motion and measure the ambient light level (illuminance), so that a central controller, referred to as the SINK node, can decide whether the room lights should be turned on or off.

1.1 Sensors choices

For the human motion detection, I've decided to go with the Passive InfraRed (PIR) motion sensor, and for the ambient light level, I've used the Photoresistor (LDR) sensor. Both can be found in the Wokwi Documentation (<https://docs.wokwi.com/>)

2 Other constraint

- Implement communication between the motion sensor and a separate central controller. The motion sensor must transmit both the motion status and the ambient light level as a single string, using one of the following formats, where ZZZ is the measured ambient light value:
 - MOTION_DETECTED-LUMINOSITY:ZZZ
 - MOTION_NOT_DETECTED-LUMINOSITY:ZZZ
- After transmitting the message, the motion sensor must enter Deep Sleep for X seconds. When the Deep Sleep interval expires, the node wakes up and resumes its normal operation, which includes sensing and transmission.

3 IoT Motion Sensor Specification (1.5pt)

I will start by explaining the code I wrote.

3.1 Include and Define

```
1 #include <esp_now.h>
2 #include <WiFi.h>
3
4 uint8_t broadcastAddr[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
5
6 esp_now_peer_info_t peerInfo;
7
8 #define PIR 18
9 #define LDR 34
10
11 #define X 4.6
12 #define C_TIME 1000000
13
14 #define GAMMA 0.7
15 #define RL10 50
```

I used both the esp_now.h and the WiFi.h library for the communication between our motion sensor and the SINK.

The broadcastAddr is a variable containing the broadcast MAC address as the SINK node address.

Also, the peerInfo is a variable containing the information needed for the communication, such as the address, the channel, and the encryption.

The PIR and LDR define indicate the number of the pin used for the respective sensor.

The X and C_TIME are needed for the deep sleep cycle. The X is calculated with the given formula.

$$\text{leader_personcode} = 106929AB$$

$$X[s] = \frac{(AB\%50 + 5)}{10}$$

In my case, $AB = 41$, so $X = 4.6s$.

Gamma and RL10 are used to calculate the LUX level with the LDR sensor.

3.2 Functions

```
17 int getLux(){
18     int aValue = analogRead(LDR);
19     float voltage = aValue / 4096.0 * 3.3;
20     float resistance = 10000.0 * voltage / (3.3 - voltage);
21     float lux_f = pow(RL10 * 1e3 * pow(10, GAMMA) / resistance, (1/GAMMA))
22     ;
23     return (int)round(lux_f);
24 }
```

3.3 Functions

```
25 String getPIRStatus(){
26     bool pir_state = digitalRead(PIR);
27     if(pir_state == HIGH){
28         return "DETECTED";
29     } else{
30         return "NOT_DETECTED";
31     }
32 }
33
34 String getMsgToSend(){
35     String msg = "MOTION_";
36     msg += getPIRStatus();
37     msg += "-LUMINOSITY:";
38     msg += getLux();
39     return msg;
40 }
```

The first function, `getLux()`, reads the analog value from the LDR sensor and converts it to lux.

For this formula, I started with the documentation on the Wokwi website and adapted the Arduino example to work with the ESP32 microcontroller.

Documentation (<https://docs.wokwi.com/parts/wokwi-photoresistor-sensor>).

Example in Arduino (<https://wokwi.com/projects/305193627138654786>).

It returns the lux value that will be sent to the SINK.

The second function, `getPIRStatus()`, will return a string containing the information about the PIR sensor. Compared to the other sensor, this was much easier to use, thanks to the documentation on the Wokwi website.

Documentation (<https://docs.wokwi.com/parts/wokwi-pir-motion-sensor>).

The last function, `getMsgToSend()`, creates the actual message that will be sent to the SINK. It simply calls the other 2 functions to read information from both sensors and returns a string containing the message.

3.4 Setup

```
42 void setup() {
43     pinMode(PIR, INPUT);
44     pinMode(LDR, INPUT);
45
46     String msg = getMsgToSend();
47
48     WiFi.mode(WIFI_STA);
49     esp_now_init();
50
51     memcpy(peerInfo.peer_addr, broadcastAddr, 6);
52     peerInfo.encrypt = false;
53     peerInfo.channel = 0;
54     esp_now_add_peer(&peerInfo);
55
56     esp_now_send(broadcastAddr, (uint8_t *)msg.c_str(), msg.length());
57     WiFi.mode(WIFI_OFF);
```

```
58
59   esp_sleep_enable_timer_wakeup(X*C_TIME);
60   esp_deep_sleep_start();
61 }
```

All of my code is in the setup

In the first lines I set both pins connected to the sensors to INPUT because we need to read data from them.

Next I create the message that the ESP32 need to send to the SINK by calling the function previously described.

After creating the message I turn on our WiFi antenna and start the esp_now protocol.

The next block will create the contact of the SINK needed for the communication. Specifying the encryption choice and the channel other then the MAC address.

After this I send the message and turn off the WiFi antenna.

Lastly, I start the sleep timer with X * C_TIME because it is in microseconds, and then the ESP32 goes in deep sleep state.

3.5 Loop

```
63 void loop() {
64 }
```

The loop is actually empty because deep sleep starts at the end of the setup and resets to the start of the setup.

3.6 Testing

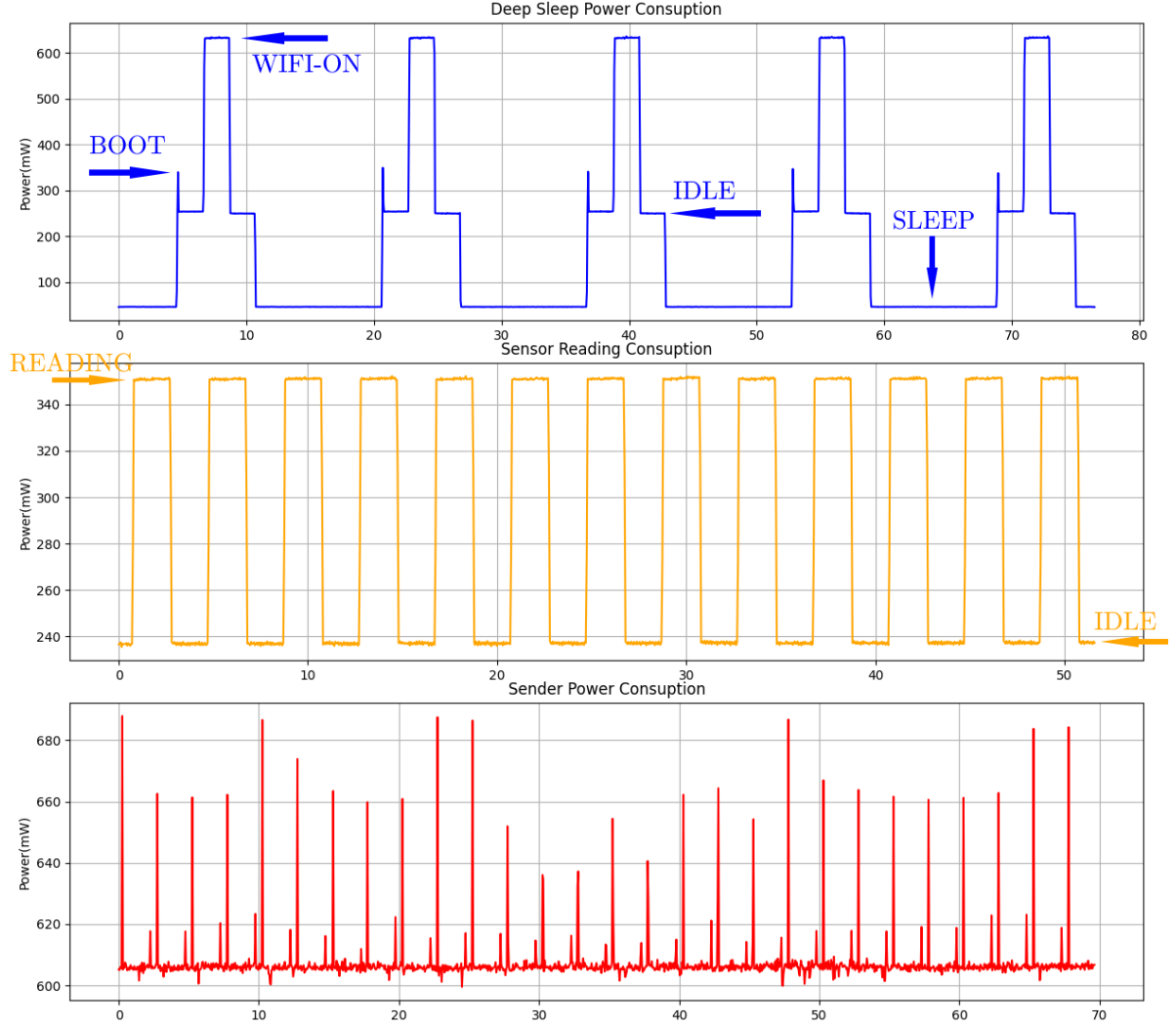
During testing, I used callbacks to ensure the message was correctly sent before the Wi-Fi module turned off and entered deep sleep. I considered adding a delay, but it wasn't necessary because for each iteration the message was correctly sent, likely because we're working on a simulator, so perhaps it could still be useful in a real-world implementation. I also used several prints to verify the correct string to send. To determine the execution times, I also used the micros() function at various points to obtain the durations of the various periods. I'll explain where these were used in more detail in the next section.

4 Energy Consumption Estimation (1 pt)

We are asked to estimate the average power consumption for each node state and the energy consumed during one full transmission cycle.

In order to do that, we are given 3 CSV files containing information about energy consumption in different states.

Using Python I obtained these graphs and the respective data.



$$P_{\text{boot}} = 342.98 \text{ mW} \simeq 343 \text{ mW}$$

$$P_{\text{idle_esp}} = 252.22 \text{ mW} \simeq 252 \text{ mW}$$

$$P_{\text{wifi_on}} = 633.23 \text{ mW} \simeq 633 \text{ mW}$$

$$P_{\text{deep_sleep}} = 45.78 \text{ mW} \simeq 46 \text{ mW}$$

$$P_{\text{reading}} = 351.09 \text{ mW} \simeq 351 \text{ mW}$$

$$P_{\text{tx_spike}} = 684.66 \text{ mW} \simeq 685 \text{ mW}$$

$$P_{\text{tx}} = P_{\text{tx_spike}} - P_{\text{wifi_on}} = 51.43 \text{ mW} \simeq 51 \text{ mW}$$

The first five values are obtained by calculating the average of the values in the specific phase. The TX value was calculated using only the eight highest values to make a conservative estimate, as the distance between our ESP32 and SINK was not specified.

From the 'sensor-read.csv' file we see that the consumption in idle is 237mW on average. Given that we have two sensors, I've decided to use the idle consumption obtained from the 'deep-sleep.csv' file, which is a little higher, with an average of 252mW in idle.

Now we need to know the battery energy, which we can calculate with this formula.

$$\text{leader_personcode} = 1069ABCD$$

$$Y[J] = (ABCD\%5000) + 15000$$

In my case $ABCD = 7341$, so $Y = 17341$ J.

To calculate consumption over an entire cycle and estimate the operating lifetime, we need to determine the cycle duration and all the different status phases. I've used ten different variables to obtain the exact time a line of code is executed with the `micros()` function.

Given that it was not possible to measure the actual booting time, I've decided to count the time used to execute the first few instructions in the code as boot, which will give a conservative estimate. I've placed one at the start of our setup and one just before the start of the deep sleep to know the time our ESP32 is awake.

Four of them have been placed before and after each of the sensors read.

Two of them have been placed before the `WiFi.mode(WIFI_STA)` and after the `WiFi.mode(WIFI_OFF)`.

The last two have been placed before and after the `esp_now_send` command.

I've used the average of more than 50 tests, the ESP32 is awake for 207410 μs , and in deep sleep for the next 4.6 s.

$$T_{\text{awake}} = 208573.59 \mu\text{s} - \text{Time our ESP32 is awake}$$

$$T_{\text{boot}} = 329.09 \mu\text{s} - \text{Time our ESP32 is booting}$$

$$T_{\text{idle_esp}} = 615.89 \mu\text{s} - \text{Time our ESP32 is in idle}$$

$$T_{\text{wifi_on}} = 207012.73 \mu\text{s} - \text{Time our ESP32 have its WiFi Antenna On}$$

$$T_{\text{reading_pir}} = 7.5 \mu\text{s} - \text{Time our ESP32 is reading from the PIR sensor}$$

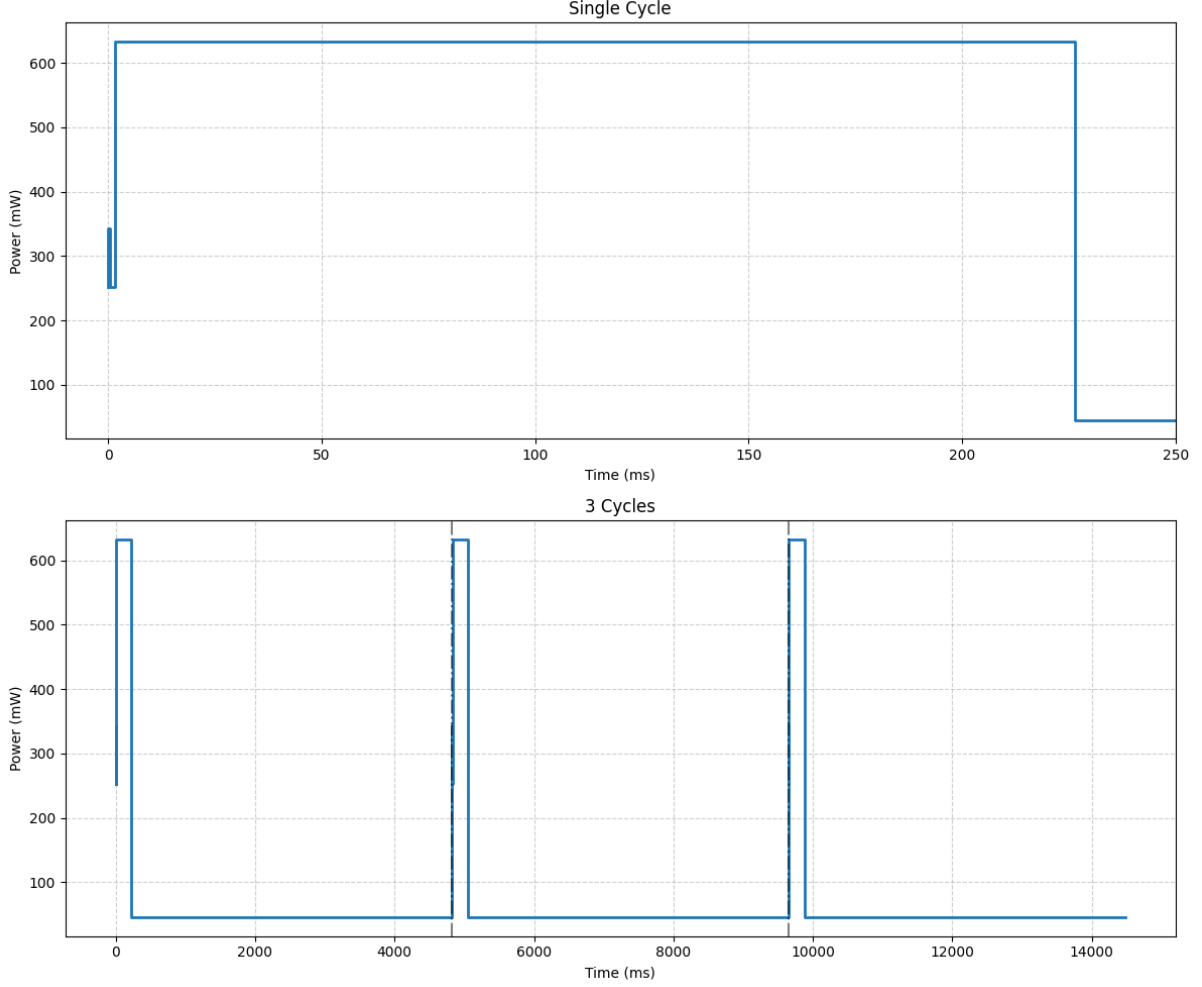
$$T_{\text{reading_ldr}} = 242.45 \mu\text{s} - \text{Time our ESP32 is reading from the LDR sensor}$$

$$T_{\text{tx}} = 80.36 \mu\text{s} - \text{Time our ESP32 is transmitting the message}$$

So the total duration of the cycle is

$$T_{\text{total}} = T_{\text{awake}} + T_{\text{deep_sleep}} = 208573.59 \mu\text{s} + 4.6 \text{ s} = 4808573.56 \mu\text{s}$$

Now we can start calculating our energy consumption.



In the first graph, we can see a boot spike at the start, followed by a brief period of idle and reading states. During that period, the program will read the sensor values and create the string to be sent to the SINK. After that, the WiFi antenna is turned on, and the message is sent to the SINK node. After turning off the WiFi antenna, the ESP32 enters deep sleep mode almost instantly for the next X seconds, then repeats the cycle. So the energy consumption for the ESP32 chip is

$$\begin{aligned}
 E_{\text{ESP}} &= T_{\text{boot}} \cdot P_{\text{boot}} + T_{\text{idle_esp}} \cdot P_{\text{idle_esp}} + (T_{\text{reading_pir}} + T_{\text{reading_ldr}}) \cdot P_{\text{reading}} + \\
 &\quad + T_{\text{wifi_on}} \cdot P_{\text{wifi_on}} + T_{\text{tx}} \cdot P_{\text{tx}} + T_{\text{deep_sleep}} \cdot P_{\text{deep_sleep}} = \\
 &= 0.329 \text{ ms} \cdot 343 \text{ mW} + 0.616 \text{ ms} \cdot 252 \text{ mW} + (0.008 \text{ ms} + 0.242 \text{ ms}) \cdot 351 \text{ mW} + \\
 &\quad + 207.013 \text{ ms} \cdot 633 \text{ mW} + 0.080 \text{ ms} \cdot 51 \text{ mW} + 4600 \text{ ms} \cdot 46 \text{ mW} = \\
 &= 342999.138 \mu\text{J} = 342.999 \text{ mJ} \simeq 343 \text{ mJ}
 \end{aligned}$$

So in total, one single cycle consumes

$$E_{\text{One_Cycle}} = 343 \text{ mJ}$$

So our battery will last for

$$N_{\text{Cycle}} = \frac{Y}{E_{\text{One_Cycle}}} = \frac{17341 \text{ J}}{343 \text{ mJ}} = 50556.851 \text{ Cycles}$$

Which results in

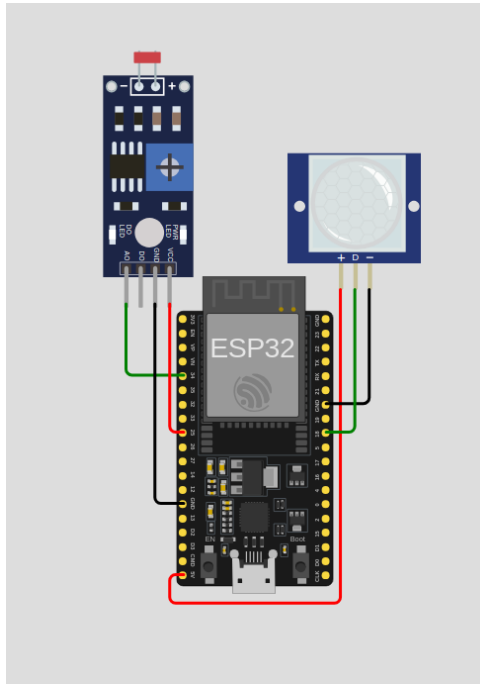
$$\begin{aligned} T_{\text{Lifetime}} &= N_{\text{Cycle}} \cdot T_{\text{total}} = \\ &= 50556.851 \text{ Cycles} \cdot 4.80857356 \text{ s} = \\ &= 243106.337 \text{ s} \simeq 4051 \text{ min} = 67 \text{ h } 31 \text{ min} = 2 \text{ d } 19 \text{ h } 31 \text{ min} \end{aligned}$$

5 Comment Results and Improvements (2.5 pt)

My first improvements regard the sensors. I was thinking of using a digital pin for supplying power and avoiding having them in an idle state all the time.

According to the specification of the PIR sensor, it takes ten to sixty seconds to warm up, so it won't be feasible to always turn it on and off.

On the other hand, the LDR sensor doesn't require any warm-up time, so we can connect it to a digital pin to manage the power supply to the sensor and only give it power when needed for the reading.



Using pin 25 instead of the 3V3, we can avoid all the idle time for the LDR sensor.

This is the change needed in the code

```
20 int getLux(){
21   digitalWrite(LDR_POWER, HIGH);
22   int aValue = analogRead(LDR);
23   digitalWrite(LDR_POWER, LOW);
24   float voltage = aValue / 4096.0 * 3.3;
25   float resistance = 10000.0 * voltage / (3.3 - voltage);
26   float lux_f = pow(RL10 * 1e3 * pow(10, GAMMA) / resistance, (1/GAMMA))
27   ;
27   return (int)round(lux_f);
28 }
```

Another improvement we can implement is related to the sleep time of our ESP32, we could remove the X seconds deep sleep and put a deep sleep only regarding the PIR sensor readings. We could put the ESP32 into deep sleep up until there is a change in the pin used for the PIR sensor.

```
65 esp_sleep_enable_ext0_wakeup(PIR, !pir_state);
```

With this, we are making the ESP32 sleep until there is a different reading in the PIR sensor. The only thing we have to check is that the PIR sensor must be set to keep the HIGH signal until there is no more movement.

It's not easy to quantify the consumption with these improvements without knowing how many times a person will enter the room.

If the room is a storage room, we may have a number of ESP32 activations in the order of tens. If we consider 50 activations per day, the average sleep time is 1728 s.

Also, the time in all of our ESP32 phases while awake doesn't change by much, given that we only added two more instructions, so we can keep the values calculated in the first part.

Our new energy consumption will be

$$\begin{aligned}
 E_{\text{ESP}} &= T_{\text{boot}} \cdot P_{\text{boot}} + T_{\text{idle_esp}} \cdot P_{\text{idle_esp}} + (T_{\text{reading_pir}} + T_{\text{reading_ldr}}) \cdot P_{\text{reading}} + \\
 &+ T_{\text{wifi_on}} \cdot P_{\text{wifi_on}} + T_{\text{tx}} \cdot P_{\text{tx}} + T_{\text{deep_sleep}} \cdot P_{\text{deep_sleep}} = \\
 &= 0.329 \text{ ms} \cdot 343 \text{ mW} + 0.616 \text{ ms} \cdot 252 \text{ mW} + (0.008 \text{ ms} + 0.242 \text{ ms}) \cdot 351 \text{ mW} + \\
 &+ 207.013 \text{ ms} \cdot 633 \text{ mW} + 0.080 \text{ ms} \cdot 685 \text{ mW} + 1728000 \text{ ms} \cdot 46 \text{ mW} = \\
 &= 79619399.138 \text{ } \mu\text{J} = 79.619 \text{ J}
 \end{aligned}$$

With a cycle duration of

$$T_{\text{total}} = T_{\text{awake}} + T_{\text{deep_sleep}} = 208573.59 \text{ } \mu\text{s} + 1728 \text{ s} = 1728.208 \text{ s}$$

So our battery will last for

$$N_{\text{Cycle}} = \frac{Y}{E_{\text{One_Cycle}}} = \frac{17341 \text{ J}}{79.619 \text{ J}} = 217.799 \text{ Cycles}$$

Which results in

$$\begin{aligned}
 T_{\text{Lifetime}} &= N_{\text{Cycle}} \cdot T_{\text{total}} = \\
 &= 217.799 \text{ Cycles} \cdot 1728.208 \text{ s} = \\
 &= 376403.307 \text{ s} \simeq 6273 \text{ min} = 104 \text{ h } 33 \text{ min} = 4 \text{ d } 8 \text{ h } 33 \text{ min}
 \end{aligned}$$

This would almost double our lifetime, but it is still not viable for a real-time implementation, considering that we will have to change the battery more than once a week. Notice that if we calculate the maximum lifetime of our ESP32, considering a full period of deep sleep we obtain

$$\begin{aligned}
 T_{\text{Max.Lifetime}} &= \frac{Y}{P_{\text{deep_sleep}}} = \frac{17341 \text{ J}}{46 \text{ mW}} = \\
 &= 376978.26 \text{ s} \simeq 6283 \text{ min} = 104 \text{ h } 43 \text{ min} = 4 \text{ d } 8 \text{ h } 43 \text{ min}
 \end{aligned}$$

So we almost maximized the lifetime of our ESP32 with those improvements.

6 Wokwi Links

Challenge 1 Project: <https://wokwi.com/projects/458932344716356609>

Challenge 1 - Optimized project: <https://wokwi.com/projects/458932676771121153>

7 Energy Trade-off in a WSN (2pt)

7.1 Q1

$$\begin{aligned}k &= 2000 \text{ bit} \\E_{\text{tx}}(k, d) &= kE_c + k\varepsilon d^2 \\E_{\text{rx}}(k) &= kE_c \\E_c &= 300 \text{ nJ/bit} \\\varepsilon &= 1 \text{ nJ/bit/m}^2\end{aligned}$$

Case A:

$$E_{\text{DIR}} = E_{\text{tx}}(2000, 40m) + E_{\text{rx}}(2000) = 4.4 \text{ mJ}$$

Case B:

$$E_{\text{HOP}} = (E_{\text{tx}}(2000, 10m) + E_{\text{rx}}(2000)) \cdot 4 = 5.6 \text{ mJ}$$

7.2 Q2

For a long-term operation I would choose Case B even if the total consumption is higher than Case A, because the consumption is divided across all of the components, so the lifetime of the system will be higher.